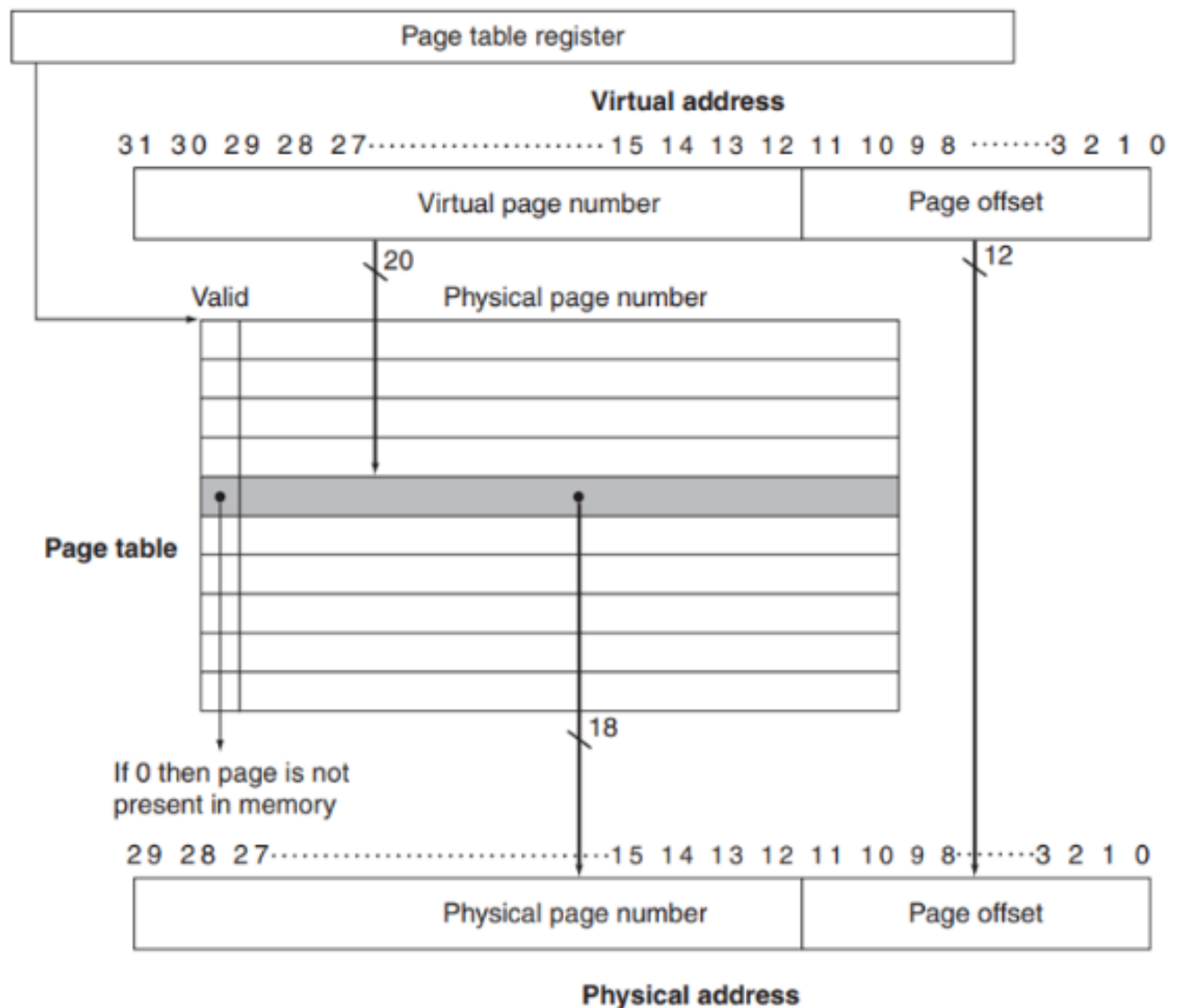


(from text Fig. 4.73)



protection and sharing

ability to create, change page tables **restricted to OS**
(how?)

status bits can include one or more bits that control
the **types of allowed access** (*writeable? executable?*)

possible to **share** physical pages (*how? what use cases?*)

making address translation fast

the problem: page tables are stored in main memory, main memory is very slow, address translation needed for *every instruction fetch and every load/store ...*

the solution: use cache(s) for page table entries on the processor chip, called TLBs (“translation-lookaside buffers”)

each entry in a TLB includes

- a page table entry (status bits plus a physical page number)
- a tag field that specifies which virtual page this entry is for (extracted from the VPN)
- in some systems, a PID or ASID (*if don't have, OS has to flush TLB on every process switch!*)

typically have separate instruction and data primary (level 1) TLBs, as well as a secondary (level 2) TLB

TLBs *much* smaller than processor caches, yet very small miss rates (typically less than 1%) (*why?*)

should processor cache lookups use *virtual* or *physical* memory addresses?

problem with using **virtual** addresses is that *each process has own mapping* to physical addresses

... but for **primary caches**, there's also a problem with using **physical** addresses ... *lookup for primary caches must be very fast!*

example illustrating one common approach

32-bit virtual addresses, **4 KiB = 2^{12} byte** pages

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

32 KiB 8-way set-associative primary cache, **$64 = 2^6$ byte** memory block size

- # of sets = **$32 \text{ KiB} / (8 \times 64 \text{ bytes}) = 64 = 2^6$**

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

since **red bits** are part of the page offset, which doesn't change, **can initiate the cache lookup in parallel** with virtual address translation

possible events when attempt to read memory
(e.g., “lw” or instruction fetch)

if **hit** in **TLB**

- can get translation into physical address from TLB, proceed with cache lookup(s)

if **miss** in **TLB**

- need to access page table of the running process, load page table entry into TLB, and then *restart the read operation*
- in some systems hardware handles, in other systems exception generated and OS handles

if **miss** in **TLB** *and* there is **no valid page table entry**

- termed a **page fault** ... **must be handled by OS**
- **invalid address?** ... if so, application terminated
- **valid address but page not yet loaded in memory?** ... if so, OS *initiates I/O* to read page in from disk ... when I/O **finally** (!) completes, OS updates page table and marks process as being runnable again